UNIVERSITÉ LAVAL

TP3 : expression différentielle avec DESeq2

Avant de débuter

DESeq2 est un package **Bioconductor** : il est déjà installé sur le serveur. Sur votre ordinateur, il s'installe avec `BiocManager::install("DESeq2")` (plusieurs minutes).

```
library(tidyverse)
library(DESeq2)
```

Le chargement de DESeq2 affiche de nombreux messages (`Attaching package...`, `The following objects are masked...`) : ce ne sont pas des erreurs, mais lisez-les. Ils signalent par exemple que `count()` et `slice()` désignent maintenant des fonctions d'autres packages (`matrixStats`, `IRanges`) : pour celles du *tidyverse*, écrivez `dplyr::count()` et `dplyr::slice()`.

Les données

Expérience `airway` (Himes *et al.*, 2014) : des cellules musculaires lisses des voies respiratoires de **4 donneurs** (4 lignées), chacune **traitée ou non** à la dexaméthasone, un corticoïde utilisé contre l'asthme. Soit 8 échantillons séquencés (RNA-seq).

Fichier	Contenu
`/home/public/stat/airway_counts.tsv`	une ligne par gène (identifiant Ensembl dans la colonne `gene`), une colonne de comptes par échantillon
`/home/public/stat/airway_sample_info.tsv`	une ligne par échantillon : `Run` (identifiant), `cell` (lignée), `dex` (`trt` traité ou `untrt` non traité), et d'autres colonnes descriptives

Rappel : les étapes

```
# 1. Une matrice de comptes (gènes en lignes) et un tableau des échantillons (échantillon
comptes <- read_tsv(...) |> column_to_rownames("gene") |> as.matrix()
infos   <- read_tsv(...) |> column_to_rownames("Run")
```

```r
# 2. L'objet DESeq2, avec le plan d'expérience
dds <- DESeqDataSetFromMatrix(countData = comptes, colData = infos, design = ~ ...)
# 3. Le test, puis les résultats
dds <- DESeq(dds)
res <- results(dds)
```

À votre tour

0. Importez les deux fichiers. Gardez les colonnes `Run`, `cell` et `dex` du second. Combien de gènes et d'échantillons ? Vérifiez que les échantillons sont **dans le même ordre** dans les deux fichiers.
1. Calculez le nombre total de lectures par échantillon (`colSums()`). Combien de gènes n'ont **aucune** lecture dans aucun échantillon ? Tracez l'histogramme de `log10(compte + 1)` pour un échantillon.
2. Transformez `dex` en facteur dont le **premier niveau** est `untrt`, et `cell` en facteur. Construisez l'objet `dds` avec le plan `~ cell + dex`, puis retirez les gènes qui ont moins de 10 lectures au total. Combien de gènes restent ?
3. Lancez `DESeq()`, puis `results()`. Que dit la **première ligne** de l'affichage de `res` ? Lancez `summary(res)`.
4. Combien de gènes ont une `padj` < 0.05 ? Combien ont en plus un |log2FoldChange| > 1 ? Combien sont induits, combien réprimés ?
5. Tracez le MA-plot (`plotMA(res)`). Transformez les comptes avec `vst(dds, blind = FALSE)` et tracez l'ACP avec `plotPCA(vsd, intgroup = c("dex", "cell"))`. Qu'est-ce qui sépare les échantillons sur PC1 ? Sur PC2 ?
6. Construisez le *volcano plot*. Listez les 10 gènes les plus **induits** (padj < 0.05, plus grand log2FoldChange). Exportez le tableau des gènes différentiellement exprimés avec `write_csv()`.

Et si… ?

Pour chaque question, **prédisez d'abord la réponse sans exécuter**, puis vérifiez dans R.

1. Et si on comptait les gènes DE avec `sum(res$padj < 0.05)`, sans `na.rm = TRUE` ? Pourquoi ? Combien de gènes ont une `padj` manquante, et pourquoi DESeq2 ne les a-t-il pas testés ?
2. Et si on utilisait `pvalue` au lieu de `padj` ? Combien de gènes « DE » ? Combien de faux positifs attendrait-on, au seuil de 5 %, si **aucun** gène ne réagissait au traitement ?
3. Et si on construisait l'objet avec `design = ~ dex` seulement : combien de gènes DE (padj < 0.05) ? Pourquoi moins ?
4. Et si les colonnes de la matrice de comptes n'étaient pas dans le même ordre que les lignes du tableau des échantillons (`comptes[, c(2, 1, 3:8)]`) ? Lisez le message.
5. Et si on n'avait pas fixé `untrt` comme premier niveau ? Que dirait la première ligne de `res` ? Que deviendrait le log2FoldChange de `ENSG00000152583` (*SPARCL1*) ?